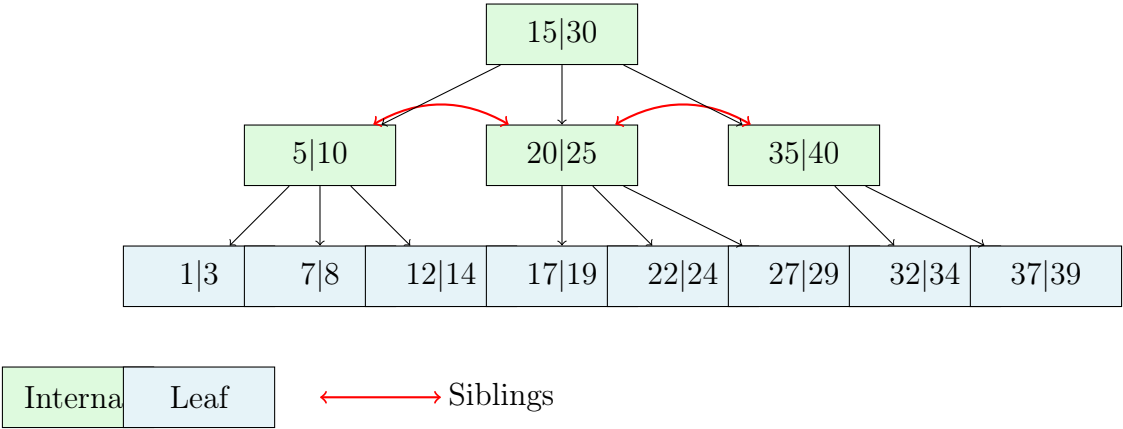


B-Tree Data Structure

Implementation Analysis and Algorithm Documentation



Filesystem Implementation

Analysis of Disk-Based C Implementation

October 17, 2025

Contents

1	Introduction	2
2	Data Structure Overview	2
2.1	BTreeNode Structure	2
2.2	Sibling Relationship Management	2
3	Core Algorithms	3
3.1	Search Algorithm	3
3.2	Insertion Algorithm	3
3.3	Deletion Algorithm	4
4	Disk Interface Integration	5
5	Implementation Functions	5
5.1	Node Management	5
5.2	Tree Operations	5
5.3	Structural Operations	5
6	Sibling Management Details	6
6.1	Split Operation Sibling Updates	6
6.2	Merge Operation Sibling Updates	6
7	Performance Characteristics	6
8	Conclusion	6

1 Introduction

B-Trees are self-balancing tree data structures that maintain sorted data and allow searches, sequential access, insertions, and deletions in logarithmic time. This document analyzes a specific C implementation of a B-Tree designed for filesystem storage systems with disk-based persistence.

The implementation features several key characteristics:

- Disk-based storage with block-level I/O operations
- Sibling relationships maintained only for internal nodes
- Dynamic node splitting and merging operations
- Support for both search and modification operations

2 Data Structure Overview

2.1 BTreeNode Structure

The core data structure is the `BTreeNode`, which represents both internal and leaf nodes:

```
1 typedef struct BTreeNode {
2     uint64_t block_number;           // Physical block number on disk
3     bool is_leaf;                    // Whether this is a leaf node
4     uint64_t key;                    // Actual key (for leaf nodes)
5     uint64_t value;                  // Associated value for key-value
6     pairs
7     uint16_t num_keys;                // Current number of keys in node
8     uint64_t keys[MAX_KEYS];         // Array of keys
9     uint64_t children[MAX_KEYS + 1]; // Child block numbers (
10    internal only)
11    uint64_t parent;                  // Parent node block number (0 if
12    root)
13    uint64_t left_sibling;            // Left sibling (internal nodes
14    only)
15    uint64_t right_sibling;           // Right sibling (internal nodes
16    only)
17 } BTreeNode;
```

2.2 Sibling Relationship Management

A critical aspect of this implementation is that **only internal nodes maintain sibling relationships**. The `left_sibling` and `right_sibling` fields are:

- Set to meaningful values only for internal nodes
- Set to 0 for leaf nodes and the root node
- Updated during split and merge operations
- Used for efficient traversal and rebalancing

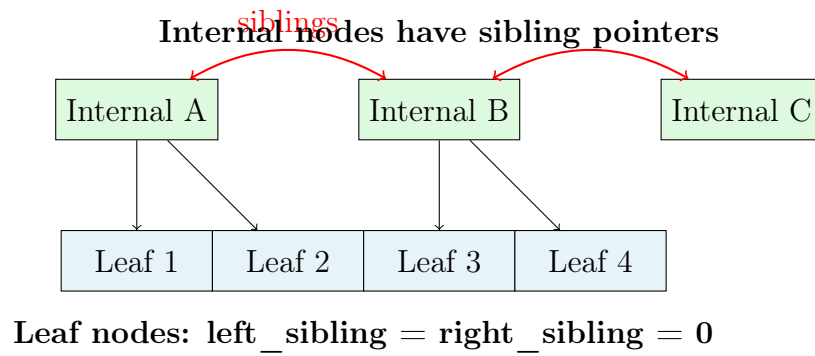
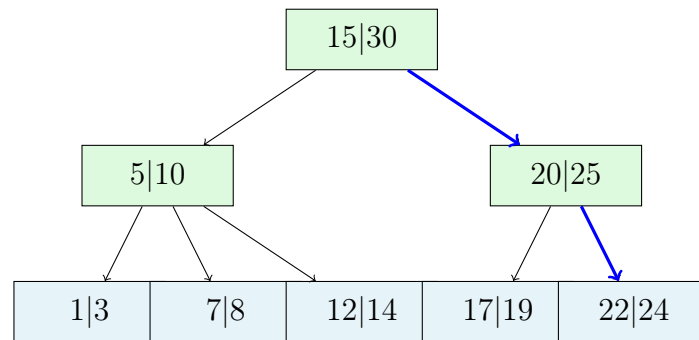


Figure 1: Sibling Relationship Structure

3 Core Algorithms

3.1 Search Algorithm

The search operation traverses the tree from root to leaf, following the appropriate child pointers based on key comparisons.



Search path for key 22

Figure 2: B-Tree Search Operation

Algorithm Steps:

1. Start at root node
2. Compare search key with node keys
3. If leaf node: return success/failure
4. If internal node: follow appropriate child pointer
5. Repeat until key found or leaf reached

3.2 Insertion Algorithm

Insertion maintains the B-Tree properties through node splitting when nodes become full.

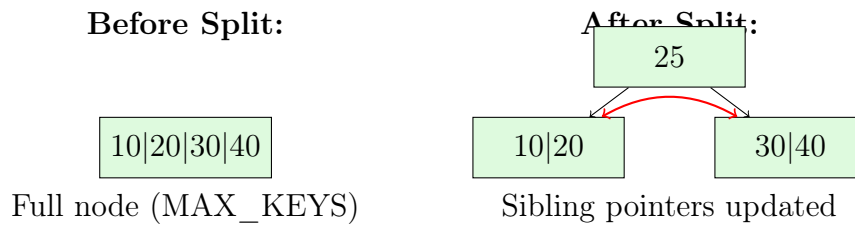


Figure 3: Node Splitting During Insertion

Key Points:

- When a node splits, sibling pointers are established between the two new internal nodes
- The maximum value of the left subtree is propagated upwards to the parent
- Leaf nodes do not maintain sibling relationships
- Root splitting creates a new root level

3.3 Deletion Algorithm

Deletion is more complex, involving borrowing from siblings or merging nodes when they become too sparse.

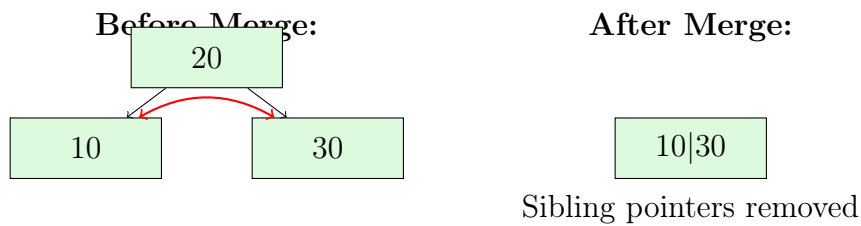


Figure 4: Node Merging During Deletion

Merge Process:

1. Identify underflowing nodes
2. Attempt to borrow from siblings (if they have extra keys)
3. If borrowing fails, merge with a sibling
4. Update sibling pointers after merge
5. Recursively handle parent node changes

4 Disk Interface Integration

The B-Tree implementation integrates with a disk interface for filesystem storage:

```
1 typedef struct DiskInterface {  
2     int disk_file;                // File handle  
3     void* disk_base;              // Memory-mapped base  
4     uint64_t total_blocks;        // Total blocks available  
5     bool is_mounted;              // Mount status  
6 } DiskInterface;
```

Key Operations:

- `disk_read_block()`: Load node from disk
- `disk_write_block()`: Persist node to disk
- `alloc_page()`: Allocate new disk block
- `free_page()`: Deallocate disk block

5 Implementation Functions

5.1 Node Management

- `btree_node_create()`: Allocate and initialize new node
- `btree_node_free()`: Deallocate node and free disk block
- `btree_node_read()`: Load node from disk block
- `btree_node_write()`: Save node to disk block

5.2 Tree Operations

- `btree_search()`: Find key in tree
- `btree_insert()`: Add new key-value pair
- `btree_delete()`: Remove key from tree
- `btree_print()`: Display tree structure

5.3 Structural Operations

- `btree_split_child()`: Split full child node
- `btree_merge_children()`: Merge sparse sibling nodes
- `btree_update_parent_keys()`: Maintain parent key consistency
- `btree_borrow_left/right()`: Redistribute keys between siblings

6 Sibling Management Details

6.1 Split Operation Sibling Updates

When an internal node splits:

1. Create two new nodes from the original
2. Set `left_sibling` and `right_sibling` between the new nodes
3. Propagate the maximum value of the left subtree upwards to the parent
4. Update any existing sibling relationships
5. Leaf nodes created during splits maintain `left_sibling = right_sibling = 0`

6.2 Merge Operation Sibling Updates

When internal nodes merge:

1. Combine keys from both nodes
2. Update sibling pointers to skip the merged node
3. Set merged node's sibling pointers to 0
4. Free the now-unused node

7 Performance Characteristics

- **Search:** $O(\log n)$ time complexity
- **Insert:** $O(\log n)$ amortized time
- **Delete:** $O(\log n)$ amortized time
- **Space:** $O(n)$ space complexity
- **Disk I/O:** Minimized through block-based operations

8 Conclusion

This B-Tree implementation provides an efficient, disk-based data structure suitable for filesystem implementations. The key design decisions include:

- Sibling relationships maintained only for internal nodes
- Disk-based persistence with block-level I/O
- Standard B-Tree balancing through splits and merges
- Support for both point queries and range operations

The restriction of sibling pointers to internal nodes simplifies the implementation while maintaining the essential properties needed for efficient tree traversal and rebalancing operations.